

AWS Employee Onboarding Management System

Prepared in the partial fulfillment of the Summer Internship Program on AWS

AT



Under the guidance of

Mrs. Bethala Sumana, APSSDC

Mr. Juttuka Lovababu, APSSDC

Submitted by

VENNA JEESHITHA, 23BQ1A12E9, VVIT GUNTUR, (Batch -1)

TANNIRU ANJALI DEVI, 23BQ1A12C8, VVIT GUNTUR, (Batch -1)

KAMBALA SRIVANI, 23BQ1A1245, VVIT GUNTUR, (Batch -1)

KOTAPATI NAVYA SRI, 23BQ1A5477, VVIT GUNTUR, (Batch -1)

DURGAM GURU HARINI, 24BQ5A1220, VVIT GUNTUR, (Batch -1)

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to all those who have contributed to the successful completion of my summer internship project at **Andhra Pradesh Skill Development Corporation (APSSDC)**. This opportunity has been an enriching and transformative experience for me, and I am truly thankful for the support, guidance, and encouragement I have received along the way.

First and foremost, I extend my sincere regards to Mrs Sumana Bethala and Mr Juttuka Lovababu, my supervisors and mentors, for providing me with valuable insights, constant guidance, and unwavering support throughout the duration of the internship. Their expertise and encouragement have been instrumental in shaping the direction of this project.

I would like to thank the entire team at **Andhra Pradesh Skill Development Corporation (APSSDC)** for fostering a collaborative and innovative environment. The camaraderie, knowledge sharing, and feedback I received from my colleagues significantly contributed to the development and success of this project.

In conclusion, I am honoured to have been a part of this internship program, and I look forward to leveraging the skills and knowledge gained to contribute positively to future endeavours.

Thank you.

Sincerely,

Venna Jeeshitha, 23BQ1A12E9, venna jeeshitha@gmail.com.
Tanniru Anjali Devi, 23BQ1A12C8, anjalidevitanniru910@gmail.com.
Kambala Srivani, 23BQ1A1245, kambalasrivani12@gmail.com.
Kotapati Navya Sri, 23BQ1A5477, kotapatinavyasri@gmail.com.
Durgam Guru Harini, 24BQ5A1220, guruharini444@gmail.com.

ABSTRACT

Employee onboarding is one of the most important processes in any organization. Traditional onboarding methods often rely on manual paperwork, spreadsheets, email communication, and physical document verification, which can be time-consuming, error-prone, and difficult to manage. As organizations grow, handling employee records, onboarding progress, and document management manually becomes increasingly inefficient.

The AWS Employee Onboarding Automation Platform is a cloud-based web application developed to automate and simplify the employee onboarding process. The system enables HR administrators to register new employees, manage employee records, monitor onboarding progress, securely upload and manage employee documents, generate reports, and maintain HR profiles through a centralized web interface. The application provides a responsive and user-friendly dashboard that improves visibility and reduces administrative effort.

The project is built using HTML, CSS, Bootstrap, and JavaScript for the frontend, while the backend is implemented using AWS Lambda with Amazon API Gateway. Employee information is securely stored in Amazon DynamoDB, uploaded documents are managed using Amazon S3, and the application is hosted using Amazon S3 Static Website Hosting with Amazon CloudFront for secure, high-performance content delivery. AWS IAM is used to manage permissions and secure access to cloud resources

The proposed system significantly reduces manual work, improves data accuracy, enhances document security, and provides a scalable serverless architecture capable of supporting organizational growth. By leveraging AWS cloud services, this offers a reliable, cost-effective, and efficient solution for modern employee onboarding, management.

TABLE OF CONTENTS

S.No	Content	Pg.No
1.	INTRODUCTION.....	5
2.	METHODOLOGY	7
3.	TECHNOLOGY STACK	10
4.	SYSTEM DESIGN / ARCHITECTURE	12
5.	IMPLEMENTATION.....	21
6.	RESULTS	24
7.	CONCLUSION	27

1. INTRODUCTION

Employee onboarding is the process of integrating newly recruited employees into an organization by completing essential formalities such as registration, document submission, verification, departmental allocation, and orientation. An efficient onboarding process helps employees adapt quickly to the organization while enabling the Human Resources (HR) department to manage employee information effectively.

In many organizations, the onboarding process is still carried out manually using paper documents, spreadsheets, and email communication. Such traditional methods often result in delayed approvals, misplaced documents, data duplication, and increased administrative effort. As the number of employees grows, maintaining accurate records and tracking onboarding progress becomes increasingly challenging.

The AWS Employee Onboarding Automation System is designed to automate and streamline the employee onboarding process using a serverless cloud architecture on Amazon Web Services (AWS). The application provides a centralized platform where HR administrators can register employees, maintain employee records, upload and manage employee documents, monitor onboarding progress, generate reports, and manage user profiles through an intuitive web interface.

The frontend of the application is developed using HTML, CSS, Bootstrap, and JavaScript, providing a responsive and user-friendly experience. The backend is implemented using AWS Lambda and Amazon API Gateway, while Amazon DynamoDB stores employee information securely. Employee documents are stored in Amazon S3, and the complete application is hosted using Amazon S3 Static Website Hosting and Amazon CloudFront for secure and high-performance content delivery.

This system minimizes manual effort, improves data accuracy, enhances document security, and provides a scalable and cost-effective solution for employee onboarding. By leveraging AWS serverless services, the system enables organizations to manage employee onboarding efficiently while reducing operational complexity.

2. METHODOLOGY

The development of the Cloud-Native Employee Onboarding Automation Platform followed a systematic and iterative software development methodology to ensure the successful implementation of a secure, scalable, and cloud-based employee onboarding solution. The methodology consisted of several phases, including requirements gathering, system design, implementation, database management, testing, deployment, and iterative refinement. Each phase contributed significantly to achieving the project's objectives while ensuring reliability, scalability, maintainability, and security.

2.1 Requirements Gathering

The project began with a comprehensive study of the employee onboarding process followed by various organizations. Information was collected through discussions with project mentors, online research, and an analysis of existing Human Resource (HR) management systems. The study revealed several limitations of traditional onboarding systems, including excessive paperwork, manual record management, spreadsheet-based tracking, email dependency, delayed onboarding, duplicate data entry, inefficient document handling, and increased administrative workload.

Based on these observations, the primary objective of the project was established as developing a cloud-native employee onboarding management platform capable of automating the complete onboarding lifecycle. The system was designed to manage employee registration, secure document storage, onboarding progress tracking, profile management, automated report generation, and email notifications while utilizing Amazon Web Services (AWS) serverless technologies to minimize infrastructure management and improve scalability.

The functional requirements identified during this phase included:

- HR administrator authentication and authorization.
- Employee registration and management.
- Secure document upload and storage.

- Employee onboarding progress tracking.
- Report generation and monitoring.
- User profile and settings management.
- Automated email notification service.
- Cloud-based deployment.

The technical requirements included developing a responsive frontend using HTML5, CSS3, Bootstrap, and JavaScript while implementing backend services using AWS Lambda, Amazon API Gateway, Amazon DynamoDB, Amazon S3, Amazon Simple Email Service (SES), AWS Identity and Access Management (IAM), Amazon CloudWatch, and Amazon CloudFront.

2.2 System Design

Following the requirements analysis, a modular system architecture was designed to separate the frontend and backend components while enabling secure communication through RESTful APIs.

The frontend was designed using HTML5, CSS3, Bootstrap, and JavaScript to provide a responsive and user-friendly interface that supports desktops, laptops, tablets, and mobile devices. Dedicated pages were developed for Login, Dashboard, Employee Management, Document Management, Reports, Settings, and Profile Management.

The backend architecture adopted a serverless computing model using AWS Lambda functions integrated with Amazon API Gateway. Each Lambda function was responsible for a specific business operation such as employee registration, profile retrieval, employee updates, document management, onboarding progress updates, report generation, and administrator authentication.

Employee information and HR administrator details were stored in Amazon DynamoDB, while employee documents were securely stored in Amazon S3. Amazon CloudFront was used as a Content Delivery Network (CDN) to improve website performance by reducing latency and accelerating content delivery. Amazon SES was integrated to automate welcome emails and onboarding completion notifications.

The modular architecture improves system scalability, maintainability, fault isolation, and future extensibility without affecting existing application components.

2.3 Implementation

The implementation phase involved developing both frontend and backend modules and integrating them using AWS cloud services.

The frontend was implemented using HTML5, CSS3, Bootstrap, and JavaScript. Responsive layouts were created to provide consistent user experience across different devices and screen sizes. JavaScript Fetch API was used to communicate with backend REST APIs exposed through Amazon API Gateway.

The backend was developed using multiple AWS Lambda functions written in Python. These functions implemented the business logic required for employee registration, employee information retrieval, profile management, secure document uploads, onboarding progress updates, report generation, and administrator authentication.

Amazon DynamoDB tables were created to store employee records and HR administrator information. Employee documents were uploaded securely to Amazon S3 using pre-signed URLs generated by Lambda functions, preventing direct public access to files.

Amazon SES was integrated to automatically send welcome emails to newly registered employees and onboarding completion notifications. Every frontend module was integrated with its corresponding backend API, ensuring seamless communication between the user interface and AWS cloud services.

2.4 Database Management

Efficient data management is an essential component of the AWS Employee Onboarding Management System. The application uses Amazon DynamoDB as its primary database because it is a fully managed, serverless, highly available, and scalable NoSQL database service.

Two DynamoDB tables were designed for the application.

HRUsers Table

The HRUsers table stores administrator account information required for authentication and profile management. The attributes include:

- Username
- Full Name
- Email Address
- Password

This table supports administrator authentication, profile retrieval, and profile updates.

Employees Table

The Employees table stores employee information required throughout the onboarding process.

The attributes include:

- Employee ID
- Employee Name
- Email Address
- Department
- Manager
- Date of Joining
- Employment Status
- Registration Progress
- Document Verification Status
- Training Status

Amazon DynamoDB provides high-performance read and write operations while automatically scaling based on application demand. Since DynamoDB is serverless, database administration tasks such as server provisioning, maintenance, patching, and backup management are handled automatically by AWS.

2.5 Testing

Comprehensive testing was conducted throughout the development lifecycle to verify the reliability, functionality, and performance of the AWS Employee Onboarding Management System.

Functional Testing

Each application module was tested independently to verify that all features functioned according to the specified requirements. The following modules were validated:

- Login
- Dashboard
- Employee Registration
- Employee Management
- Document Upload
- Reports
- Profile Management
- Settings
- Email Notifications

API Testing

REST APIs developed using Amazon API Gateway and AWS Lambda were tested to ensure accurate request handling, proper response generation, successful database interactions, and appropriate error handling.

Database Testing

Amazon DynamoDB tables were tested to verify successful insertion, retrieval, updating, and deletion of employee records while maintaining data consistency

Cloud Testing

The deployed application hosted on Amazon S3 and distributed through Amazon CloudFront was

tested using multiple web browsers and devices to verify accessibility, responsiveness, secure HTTPS communication, and consistent performance.

User Interface Testing

The graphical user interface was evaluated for usability, navigation, responsiveness, visual consistency, and overall user experience. The testing process confirmed that all modules performed correctly and satisfied the functional and non-functional requirements of the project.

2.6 Deployment

Following successful implementation and testing, the AWS Employee Onboarding Management System was deployed on Amazon Web Services (AWS).

The frontend application, consisting of HTML, CSS, JavaScript, Bootstrap, images, and other static resources, was uploaded to an Amazon S3 bucket configured for static website hosting.

Amazon CloudFront was configured as a Content Delivery Network (CDN) to distribute website content globally, reducing latency and improving page loading speed.

Amazon API Gateway was deployed to securely expose backend AWS Lambda functions as RESTful APIs. The Lambda functions were integrated with Amazon DynamoDB and Amazon S3 to perform all backend operations.

AWS Identity and Access Management (IAM) roles were configured to enforce the principle of least privilege by granting only the required permissions to each AWS service. Amazon CloudWatch was enabled to monitor Lambda execution logs, track application performance, and assist in troubleshooting operational issues.

The deployed architecture provides a highly available, secure, scalable, and cost-effective cloud-hosted solution capable of efficiently serving users from any geographical location.

2.7 Iterative Refinement

The AWS Employee Onboarding Management System was continuously refined throughout the development process based on testing results, mentor feedback, and implementation challenges.

Several improvements were introduced during the iterative development process, including:

- Enhanced dashboard layout and visualization.
- Improved employee search and filtering functionality.
- Secure document upload using Amazon S3 pre-signed URLs.
- Enhanced administrator profile management.
- Improved settings page functionality.
- Responsive user interface optimization.
- Better API response validation and error handling.
- More informative user error messages.
- Secure deployment through Amazon CloudFront.
- Automated email notifications using Amazon SES.

The iterative development approach significantly improved the application's performance, usability, maintainability, scalability, and security. Furthermore, the modular cloud-native architecture facilitates future enhancements such as employee self-service portals, role-based access control (RBAC), attendance management, payroll integration, analytics dashboards, mobile application support, and integration with enterprise HR systems.

3. TECHNOLOGY STACK

The AWS – Employee Onboarding Management System was developed using modern web technologies and Amazon Web Services (AWS). The application follows a serverless architecture in which the frontend, backend, database, storage, and notification services are managed by AWS. This approach provides scalability, security, reliability, and cost-effectiveness while eliminating the need to manage physical servers.

The following technologies and services were used throughout the development of the project.

3.1 Cloud Platform

Amazon Web Services (AWS)

Amazon Web Services (AWS) is the cloud platform used to develop, deploy, and manage the entire Employee Onboarding Management System. AWS provides fully managed services that simplify application development while ensuring high availability, scalability, and security.

The project uses AWS serverless services to host the frontend, execute backend logic, store employee information and documents, send email notifications, and monitor application performance.

3.2 AWS Services

Amazon S3

Amazon Simple Storage Service (Amazon S3) is used for hosting the frontend application and storing employee documents uploaded during the onboarding process.

The static website files such as HTML, CSS, JavaScript, images, and other assets are stored in an S3 bucket. Employee documents are also securely stored in a separate S3 bucket.

Features

- Static website hosting

- Secure document storage
- High durability
- Scalable cloud storage
- Cost-effective storage solution

Amazon CloudFront

Amazon CloudFront is used as a Content Delivery Network (CDN) to distribute the frontend application with low latency and secure HTTPS communication.

CloudFront retrieves website files from Amazon S3 and caches them at edge locations, resulting in faster page loading and improved user experience.

Features

- Fast content delivery
- HTTPS support
- Reduced latency
- Global content distribution
- Improved website performance

Amazon API Gateway

Amazon API Gateway acts as the communication layer between the frontend and backend.

Whenever an HR administrator performs an operation such as employee registration, document upload, or report generation, API Gateway receives the HTTP request and invokes the corresponding AWS Lambda function.

Features

- REST API management

- Secure request handling
- Integration with AWS Lambda
- High scalability
- Easy API deployment

AWS Lambda

AWS Lambda is used to implement the backend logic of the application.

Each functionality of the application is implemented as an independent Lambda function, including administrator login, employee management, document operations, report generation, profile management, and email notifications.

The serverless execution model eliminates the need to manage servers while automatically scaling based on incoming requests.

Features

- Serverless execution
- Automatic scaling
- Cost-efficient computing
- Event-driven architecture
- Easy integration with AWS services

Amazon DynamoDB

Amazon DynamoDB is the NoSQL database used to store administrator credentials and employee information.

The HRUsers table stores administrator details, while the Employees table stores employee records and onboarding progress.

Features

- Fully managed NoSQL database
- High availability
- Fast data retrieval
- Automatic scaling
- Secure data storage

Amazon Simple Email Service (SES)

Amazon SES is integrated into the application to send automated email notifications.

Whenever a new employee is added to the system, a welcome email containing employee details is automatically sent to the employee's registered email address.

Features

- Automated email delivery
- Reliable communication
- High email deliverability
- Scalable email service
- Easy AWS integration

AWS Identity and Access Management (IAM)

AWS IAM is used to securely manage permissions between AWS services.

IAM roles and policies ensure that Lambda functions can access only the required AWS resources, following the principle of least privilege.

Features

- Secure access control
- Role-based permissions
- Identity management
- Fine-grained access policies
- Improved application security

Amazon CloudWatch

Amazon CloudWatch is used to monitor application performance and collect execution logs from AWS Lambda functions.

It helps developers identify errors, monitor application health, and troubleshoot issues during development and deployment.

Features

- Log monitoring
- Performance metrics
- Error tracking
- Resource monitoring
- Operational insights

3.3 Frontend Technologies

The frontend of this application is developed using modern web technologies.

HTML5 is used to create the structure of web pages.

CSS3 is used to design responsive and visually appealing user interfaces.

Bootstrap provides responsive layouts, navigation bars, forms, buttons, and reusable components.

JavaScript handles client-side validation, dynamic content updates, and communication with backend REST APIs.

3.4 Backend Technology

The backend is implemented using **Python**.

Python is used to develop AWS Lambda functions that perform employee registration, document handling, report generation, onboarding progress tracking, and email notification.

The AWS SDK for Python (Boto3) enables seamless interaction with AWS services such as DynamoDB, S3, SES, and CloudWatch.

3.5 Data Format

The application uses **JSON (JavaScript Object Notation)** as the primary data exchange format.

JSON is used for transmitting employee information between the frontend, API Gateway, and Lambda functions. It provides a lightweight and easy-to-parse format for REST API communication.

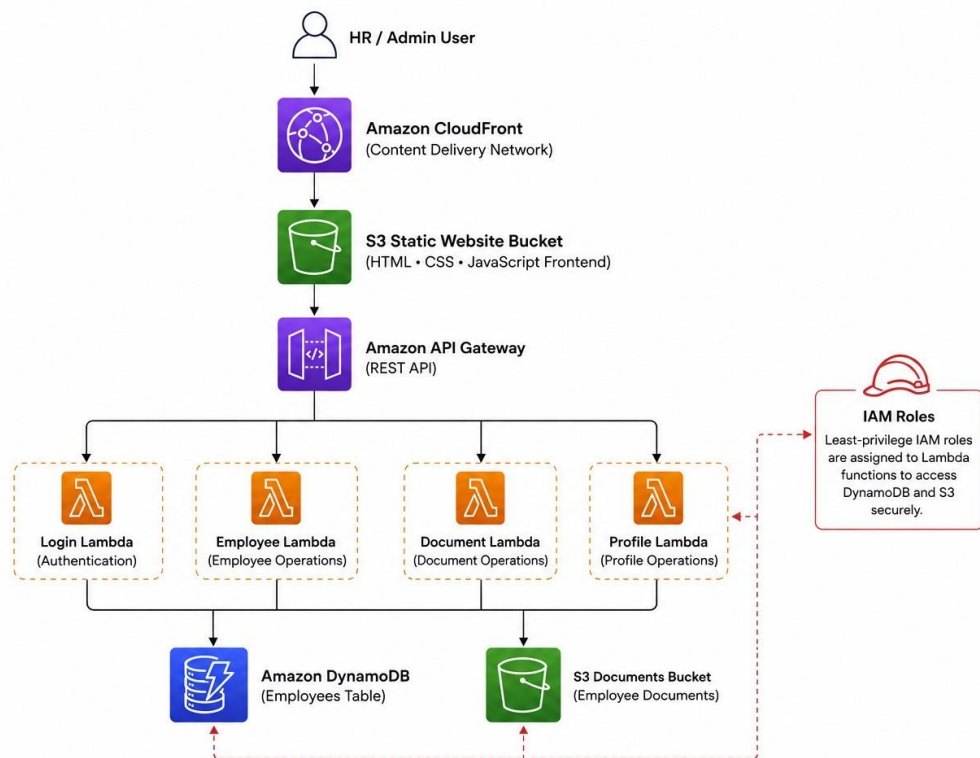
3.6 Security and Monitoring

Security is implemented using AWS IAM roles and policies, ensuring that only authorized services can access AWS resources.

Amazon CloudWatch continuously monitors Lambda execution logs and application performance, helping identify runtime errors and improve system reliability

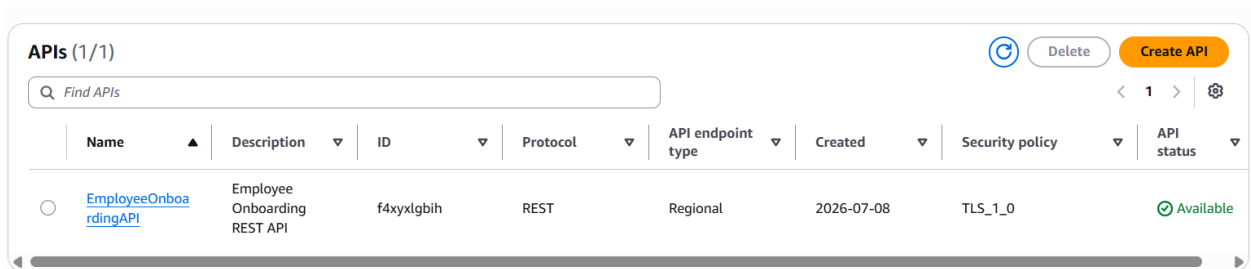
4.SYSTEM DESIGN / ARCHITECTURE

The AWS – Employee Onboarding Management System is designed as a cloud-native, serverless application that automates the employee onboarding process. The system follows a modular architecture where each component performs a specific task while interacting securely with other AWS services. The architecture emphasizes scalability, security, maintainability, and cost-effectiveness by eliminating the need to manage traditional servers.



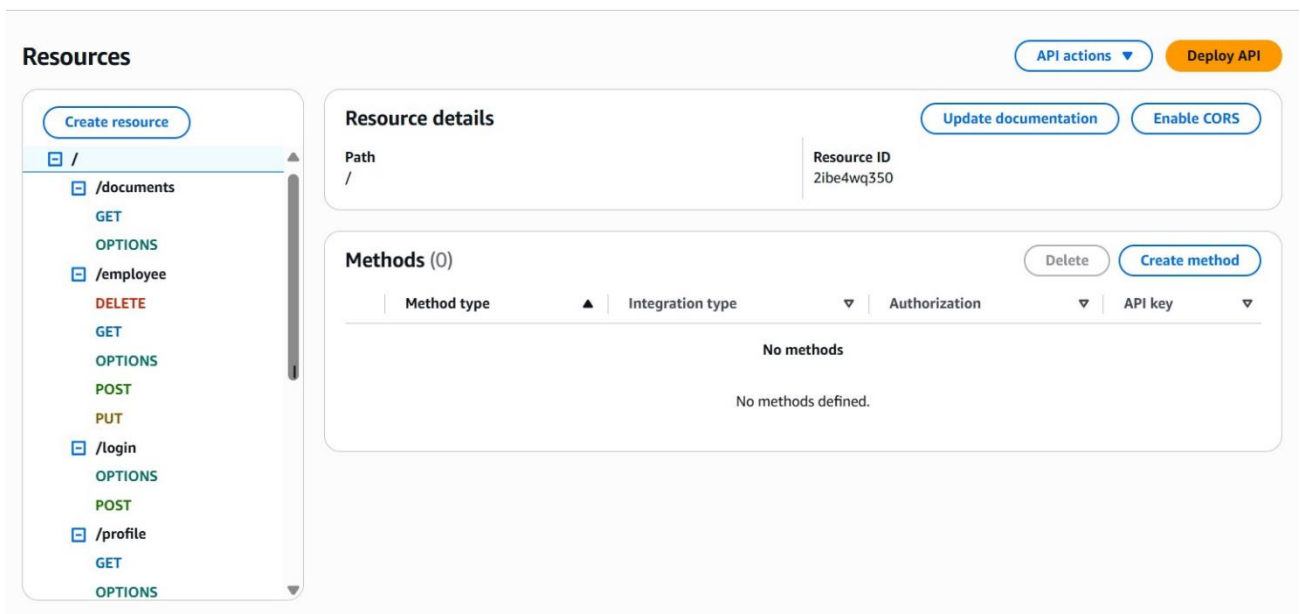
4.1. API Layer – Amazon API Gateway

Amazon API Gateway acts as the communication interface between the frontend and backend components. Whenever an HR administrator performs operations such as employee registration, employee search, document upload, profile update, or report generation, the frontend sends an HTTP request to Amazon API Gateway. API Gateway validates the request and invokes the appropriate AWS Lambda function for processing.



Name	Description	ID	Protocol	API endpoint type	Created	Security policy	API status
EmployeeOnboardingAPI	Employee Onboarding REST API	f4xyxlgbih	REST	Regional	2026-07-08	TLS_1_0	Available

Figure 1.API



Path	Resource ID
/	2ibe4wq350

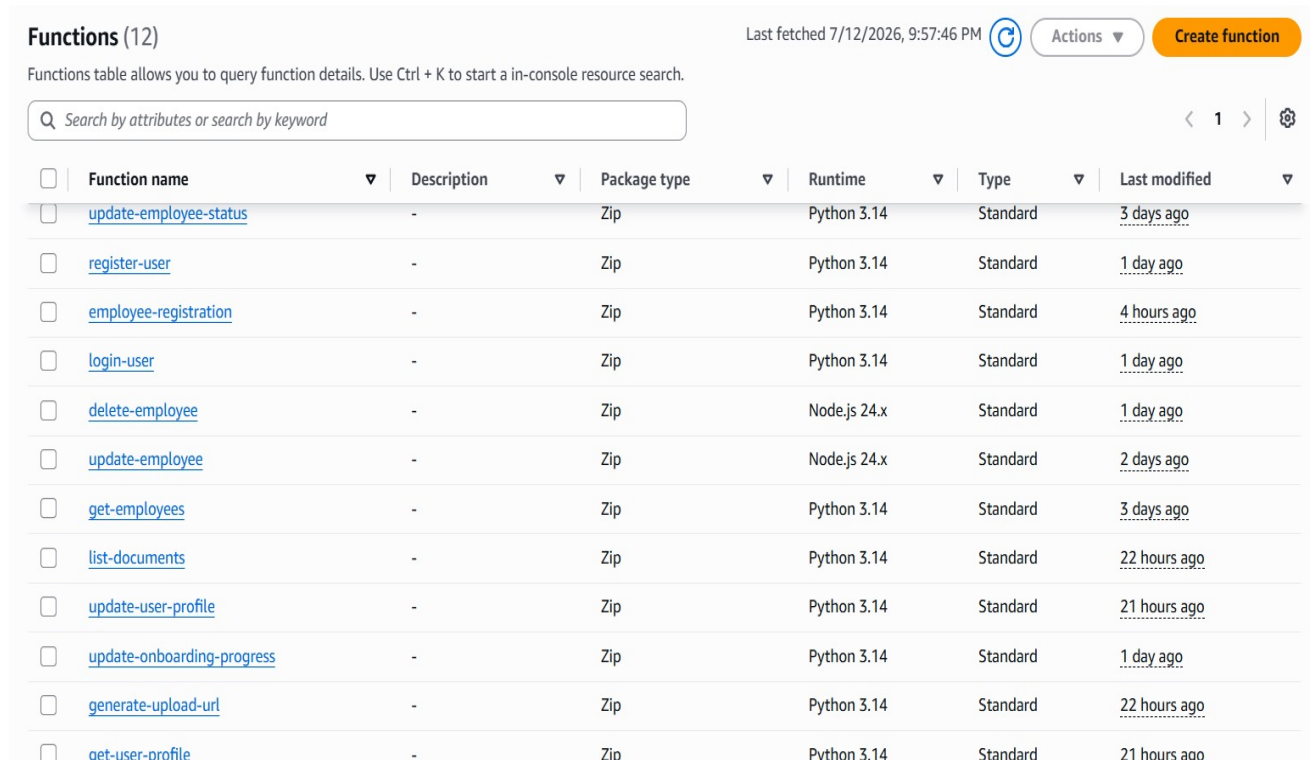
Method type	Integration type	Authorization	API key
No methods defined.			

Figure 2.Resources in AP

4.2. Application Layer – AWS Lambda

The business logic of the AWS application is implemented using **AWS Lambda**.

Each module of the application is developed as an independent Lambda function to improve modularity and maintainability. These Lambda functions execute only when invoked by API Gateway, making the application completely serverless.



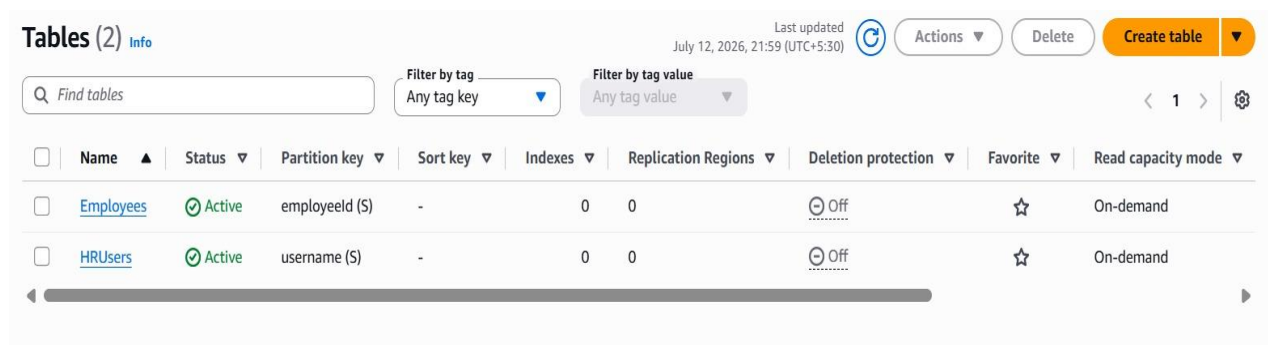
The screenshot shows the AWS Lambda console with a list of 12 functions. The table includes columns for Function name, Description, Package type, Runtime, Type, and Last modified. The functions are listed in descending order of last modified time.

Function name	Description	Package type	Runtime	Type	Last modified
update-employee-status	-	Zip	Python 3.14	Standard	3 days ago
register-user	-	Zip	Python 3.14	Standard	1 day ago
employee-registration	-	Zip	Python 3.14	Standard	4 hours ago
login-user	-	Zip	Python 3.14	Standard	1 day ago
delete-employee	-	Zip	Node.js 24.x	Standard	1 day ago
update-employee	-	Zip	Node.js 24.x	Standard	2 days ago
get-employees	-	Zip	Python 3.14	Standard	3 days ago
list-documents	-	Zip	Python 3.14	Standard	22 hours ago
update-user-profile	-	Zip	Python 3.14	Standard	21 hours ago
update-onboarding-progress	-	Zip	Python 3.14	Standard	1 day ago
generate-upload-url	-	Zip	Python 3.14	Standard	22 hours ago
get-user-profile	-	Zip	Python 3.14	Standard	21 hours ago

Figure 3- AWS Lambda functions

4.3. Database Layer – Amazon DynamoDB

Amazon DynamoDB is used as the primary database for storing application data. The project uses two DynamoDB tables and those are represented in the below picture.



The screenshot shows the Amazon DynamoDB console with two tables listed: Employees and HRUsers. The table includes columns for Name, Status, Partition key, Sort key, Indexes, Replication Regions, Deletion protection, Favorite, and Read capacity mode.

Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection	Favorite	Read capacity mode
Employees	Active	employeeid (S)	-	0	0	Off	☆	On-demand
HRUsers	Active	username (S)	-	0	0	Off	☆	On-demand

Figure 4- dynamodb tables

4.4. Document Storage – Amazon S3

Amazon Simple Storage Service (Amazon S3) is used for hosting the frontend application and storing employee documents securely. Amazon S3 provides highly durable, secure, and scalable cloud storage suitable for both static website hosting and document management.

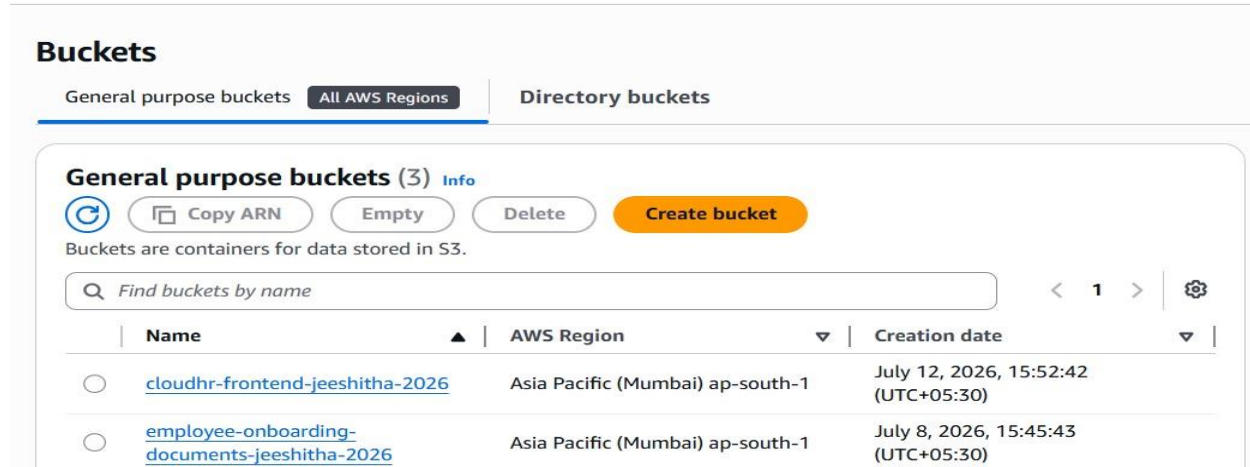


Figure 11-Amazon S3 Buckets

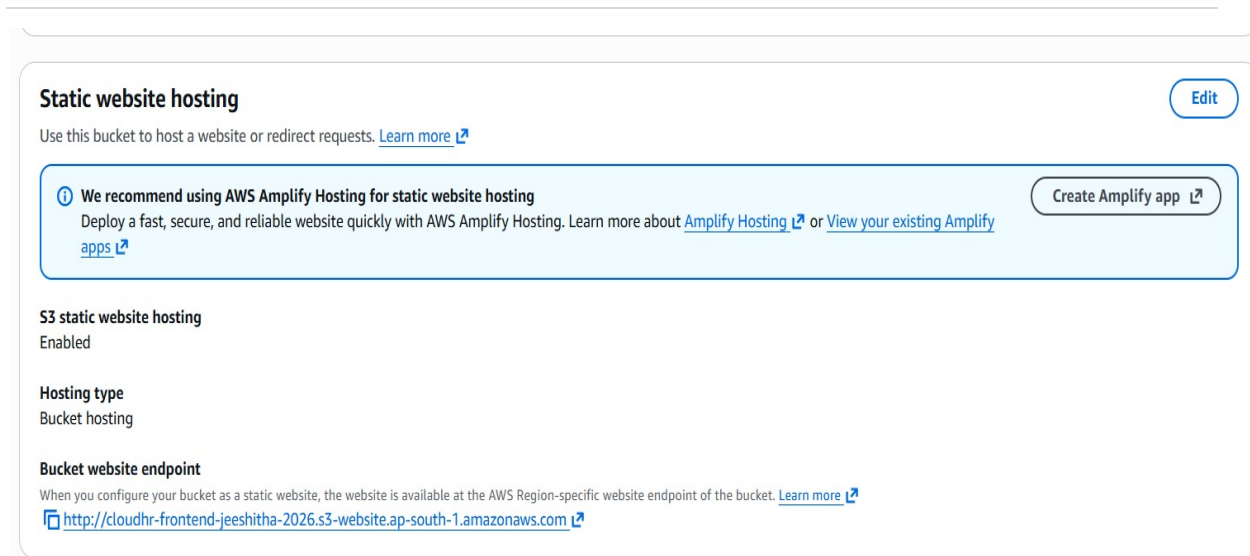


Figure 5-Static Website Hosting

4.5. Email Notification – Amazon SES

Amazon Simple Email Service (SES) is integrated into the application to automate employee communication. Whenever a new employee is registered successfully, the Employee Registration Lambda function automatically sends a personalized welcome email containing employee details such as Employee ID, Department, Manager, and Joining Date.



Figure 6-Email Notification

4.6. Security – AWS IAM

AWS Identity and Access Management (IAM) is used to manage authentication and authorization between AWS services. IAM roles are assigned to Lambda functions to allow secure access to DynamoDB, Amazon S3, Amazon SES, and CloudWatch. The project follows the Principle of Least Privilege, ensuring that each service receives only the permissions required to perform its operations.

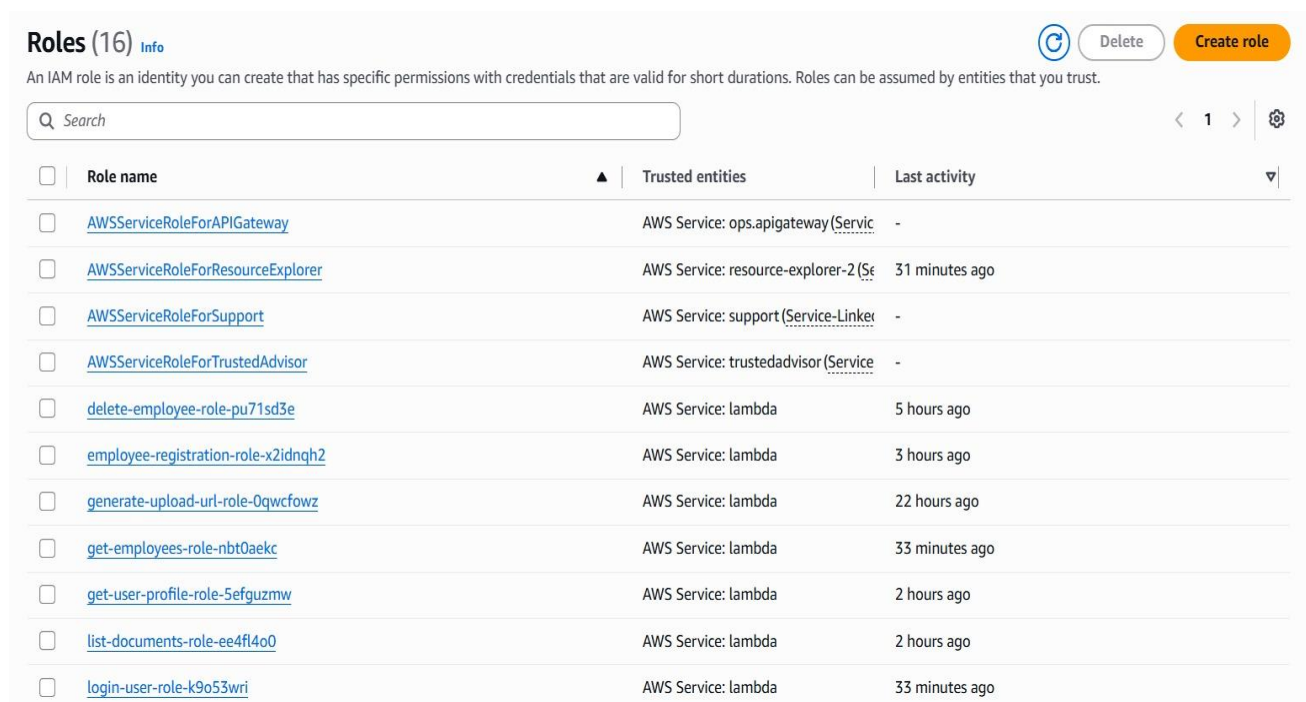


Figure 7 – IAM Roles

4.6. Monitoring – Amazon CloudWatch

Amazon CloudWatch is used to monitor application performance and record execution logs generated by AWS Lambda functions. CloudWatch helps developers identify runtime errors,

monitor request execution, and troubleshoot issues encountered during development and deployment. It also provides valuable insights into application performance and resource utilization.

Log groups (12)

By default, we only load up to 10,000 log groups.

☒ Exact match

< 1 >

☐	Log group	Log class	Anomaly d...	Deletio...	Bearer ...	Data ...	Sensi...	Retenti...
☐	/aws/lambda/delete-employee	Standard	Configure	Off	Off	-	-	Never exp...
☐	/aws/lambda/employee-registration	Standard	Configure	Off	Off	-	-	Never exp...
☐	/aws/lambda/generate-upload-url	Standard	Configure	Off	Off	-	-	Never exp...
☐	/aws/lambda/get-employees	Standard	Configure	Off	Off	-	-	Never exp...
☐	/aws/lambda/get-user-profile	Standard	Configure	Off	Off	-	-	Never exp...
☐	/aws/lambda/list-documents	Standard	Configure	Off	Off	-	-	Never exp...
☐	/aws/lambda/login-user	Standard	Configure	Off	Off	-	-	Never exp...
☐	/aws/lambda/register-user	Standard	Configure	Off	Off	-	-	Never exp...
☐	/aws/lambda/update-employee	Standard	Configure	Off	Off	-	-	Never exp...
☐	/aws/lambda/update-employee-status	Standard	Configure	Off	Off	-	-	Never exp...
☐	/aws/lambda/update-onboarding-progress	Standard	Configure	Off	Off	-	-	Never exp...

Figure 7 – Logs in CloudWatch

5. IMPLEMENTATION

Implementation is the phase in which the proposed system design is converted into a fully functional application. The AWS – Employee Onboarding Management System was developed using modern web technologies and Amazon Web Services (AWS). The application follows a serverless architecture that enables efficient employee onboarding without the need to maintain dedicated servers.

5.1. Home Module

The Home Module serves as the landing page of the AWS application. It provides a brief introduction to the Employee Onboarding Management System and allows administrators to navigate to the login page. The page includes a responsive navigation bar, project information, feature highlights, and a footer for a professional appearance.

The frontend of this module is hosted on Amazon S3 and delivered securely through Amazon CloudFront.

Features

- Responsive home page
- Navigation menu
- Project overview
- Feature highlights
- Footer section

5.2. Login Module

The Login Module is the entry point of the application. It authenticates HR administrators using credentials stored in the HRUsers table of Amazon DynamoDB.

The administrator enters the username and password through the login page. The frontend sends the credentials to Amazon API Gateway, which invokes the Login Lambda function. The Lambda function validates the credentials and returns the authentication result. Upon successful login, the administrator is redirected to the dashboard.

Features

- Secure administrator authentication
- Credential validation
- Session management
- Restricted access
- Logout functionality

5.4 Dashboard Module

The Dashboard provides an overview of the employee onboarding process. It acts as the central control panel for the HR administrator by displaying quick access to all application modules.

The dashboard retrieves employee information dynamically from Amazon DynamoDB using AWS Lambda functions and displays onboarding statistics.

Features

- Employee summary
- Navigation to all modules
- Dynamic data display
- User-friendly interface
- Centralized management

5.5 Employee Management Module

The Employee Management Module allows HR administrators to manage employee records efficiently. The administrator can register new employees by entering employee details such as Employee ID, Name, Email Address, Department, Manager, Joining Date, and Status. The

entered information is sent to AWS Lambda through Amazon API Gateway and stored in the Employees table of Amazon DynamoDB.

The module also supports updating employee information, deleting employee records, searching employees, and tracking onboarding progress.

Features

- Add employee
- View employee details
- Update employee information
- Delete employee
- Search employees
- Track onboarding progress

5.6 Document Management Module

The Document Management Module enables secure storage and management of employee documents. The administrator uploads employee documents through the web interface. The documents are transferred to AWS Lambda using Amazon API Gateway and stored securely in Amazon S3.

The module allows administrators to view uploaded documents whenever required.

Features

- Upload employee documents
- Secure cloud storage
- View uploaded documents
- Organized document management
- Reliable storage using Amazon S3

5.7 Reports Module

The Reports Module provides a summary of employee onboarding activities.

The module retrieves employee information from Amazon DynamoDB through AWS Lambda and presents it in a structured format. It also allows administrators to download employee information in CSV format for future reference.

The reports assist HR administrators in monitoring employee onboarding progress.

Features

- Employee report generation
- CSV export
- Employee statistics
- Dynamic report generation
- Easy monitoring

5.8 Profile and Settings Module

The Profile and Settings Module enables administrators to manage their personal account information.

The administrator can view profile details and update information such as name, email address, username, and password. Updated information is securely stored in the HRUsers table of Amazon DynamoDB.

Features

- View administrator profile
- Update profile information
- Change password
- Secure account management

5.9 Email Notification Module

The Email Notification Module automates communication between the HR administrator and employees. When a new employee is successfully registered, the Employee Registration Lambda function automatically invokes Amazon SES to send a personalized welcome email to the employee's registered email address. The email contains employee details including Employee ID, Department, Reporting Manager, and Joining Date.

Automating this process reduces manual effort and ensures timely communication with employees.

Features

- Welcome email notification
- Automated email delivery
- Personalized email content
- Reliable communication
- Integration with Amazon SES

5.10 Testing and Deployment

After implementing all modules, the application was tested to verify the functionality of each feature.

Different test cases were performed for administrator login, employee registration, employee management, document upload, report generation, profile update, and email notification. The frontend application was hosted on Amazon S3 and distributed through Amazon CloudFront, while backend services were deployed using AWS Lambda and Amazon API Gateway.

6. RESULTS

The implementation of the AWS – Employee Onboarding Management System yielded successful results that satisfy the objectives defined during the project planning phase. The developed application was tested under different scenarios to verify its functionality, reliability, scalability, and security. The successful integration of Amazon Web Services (AWS) enabled the application to automate employee onboarding activities efficiently while reducing manual effort. The following results demonstrate the performance and effectiveness of the implemented system.

6.1 Employee Registration and Data Management

The Employee Registration module successfully stores employee information in Amazon DynamoDB whenever the HR administrator registers a new employee. The system validates the entered information before storing it in the database, ensuring that employee records are maintained accurately and consistently.

The implemented system supports employee registration, modification of employee details, employee search, and deletion of records. All operations were performed successfully without affecting database consistency.

Key Results

- Successful employee registration.
- Accurate storage of employee information in DynamoDB.
- Quick retrieval and updating of employee records.
- Reliable employee management operations.

6.2 Automated Email Notifications

The application integrates Amazon Simple Email Service (SES) to automate communication with newly registered employees.

Whenever an employee is successfully added to the system, AWS Lambda automatically invokes Amazon SES to send a personalized welcome email containing employee information such as Employee ID, Department, Reporting Manager, and Joining Date.

This automation eliminates manual communication and ensures that employees receive onboarding information immediately after registration.

Key Results

- Automatic welcome email delivery.
- Reliable email communication using Amazon SES.
- Immediate notification after successful employee registration.
- Reduced manual effort for HR administrators.

6.3 Secure Cloud Storage

The application successfully stores employee-related documents in Amazon S3.

Employee documents uploaded through the web interface are securely transferred using AWS Lambda and stored in Amazon S3. The documents remain available for future retrieval whenever required.

Testing confirmed successful upload and storage of employee documents while maintaining data integrity.

Key Results

- Secure document storage.
- Successful upload and retrieval of employee documents.
- Reliable cloud-based storage.
- Improved document management.

6.4 Serverless Processing and API Integration

This application uses Amazon API Gateway and AWS Lambda to process all user requests.

Whenever the administrator performs an operation such as employee registration, document upload, profile update, or report generation, API Gateway successfully invokes the corresponding Lambda function. Each Lambda function executes independently and communicates with the required AWS services.

Testing confirmed successful communication between the frontend, API Gateway, Lambda, DynamoDB, Amazon S3, and Amazon SES.

Key Results

- Successful API communication.
- Reliable Lambda execution.
- Fast serverless processing.
- Smooth integration among AWS services.

6.5 Fast and Secure Web Application Delivery

The frontend application is hosted using Amazon S3 Static Website Hosting and distributed through Amazon CloudFront. CloudFront delivers website content through global edge locations, reducing page loading time while providing secure HTTPS communication. The website loaded successfully across multiple browsers and maintained consistency.

Key Results

- Fast webpage loading.
- Secure HTTPS communication.
- Improved user experience.
- Reliable frontend hosting.

6.6 Security and Monitoring

Security within the application is managed using AWS Identity and Access Management IAM roles and policies provide secure access between AWS services, ensuring that Only authorized Lambda functions can access DynamoDB, Amazon S3, Amazon SES, and CloudWatch.

Amazon CloudWatch continuously monitors Lambda execution and records application logs. During testing, CloudWatch successfully captured execution details, runtime errors, and API requests, making troubleshooting easier.

Key Results

- Secure role-based access control.
- Successful monitoring through CloudWatch.
- Reliable execution logging.
- Improved application security.

6.7 System Performance and Scalability

The serverless architecture enables the application to scale automatically according to incoming requests without requiring manual server management.

The system demonstrated stable performance while executing employee registration, document upload, report generation, and email notification simultaneously. The use of managed AWS services reduced operational overhead while maintaining high availability.

The modular implementation allows future enhancements without affecting existing functionalities.

Key Results

- Automatic scalability.

- Stable system performance.
 - Reduced infrastructure maintenance.
 - Efficient resource utilization.
-

7. CONCLUSION

The **AWS – Employee Onboarding Management System** was successfully designed and implemented as a cloud-based, serverless application using **Amazon Web Services (AWS)**. The primary objective of the project was to automate the employee onboarding process by replacing manual activities with an efficient, secure, and scalable digital solution.

The developed application enables HR administrators to register employees, manage employee records, upload and store documents securely, monitor onboarding progress, generate reports, and send automated welcome email notifications. The integration of **Amazon S3, Amazon CloudFront, Amazon API Gateway, AWS Lambda, Amazon DynamoDB, Amazon SES, AWS IAM, and Amazon CloudWatch** resulted in a reliable serverless architecture that provides high availability, security, and improved performance while minimizing infrastructure management.

The system successfully achieved all the objectives defined at the beginning of the project. Employee information is managed efficiently, documents are securely stored in the cloud, and automated email notifications improve communication between the HR department and employees. The modular design of the application makes it easy to maintain, update, and extend with additional features in the future.

Overall, the AWS Employee Onboarding Management System demonstrates how cloud computing and serverless technologies can simplify HR operations, reduce manual effort, improve data accuracy, and enhance the overall onboarding experience. The project provides a practical and scalable solution that can be adopted by organizations to streamline employee onboarding while taking advantage of modern cloud technologies.
